CHAPTER

10

取得與標註 YOLO 訓練資料：LabelImg

- 10-1　認識 Ultralytics 的 YOLO
- 10-2　Thonny Python IDE 的基本使用
- 10-3　取得訓練 YOLO 模型的圖檔資料
- 10-4　使用 LabelImg 標註圖檔建立訓練資料
- 10-5　整合應用：在 Node-RED 顯示標註圖檔

10-1 認識 Ultralytics 的 YOLO

YOLO 原名 You only look once，英文的意思就是只需看一次，就可以快速且準確的偵測出影像或視訊影格等數位內容中的多種物體。

簡單的說，YOLO 可以在單一影像中識別出多種不同分類的多個物體，而且，不只能夠識別出物體分類，還能夠偵測出物體所在的邊界框，知道這些物體在哪裡。

10-1-1 YOLO 物體偵測的深度學習演算法

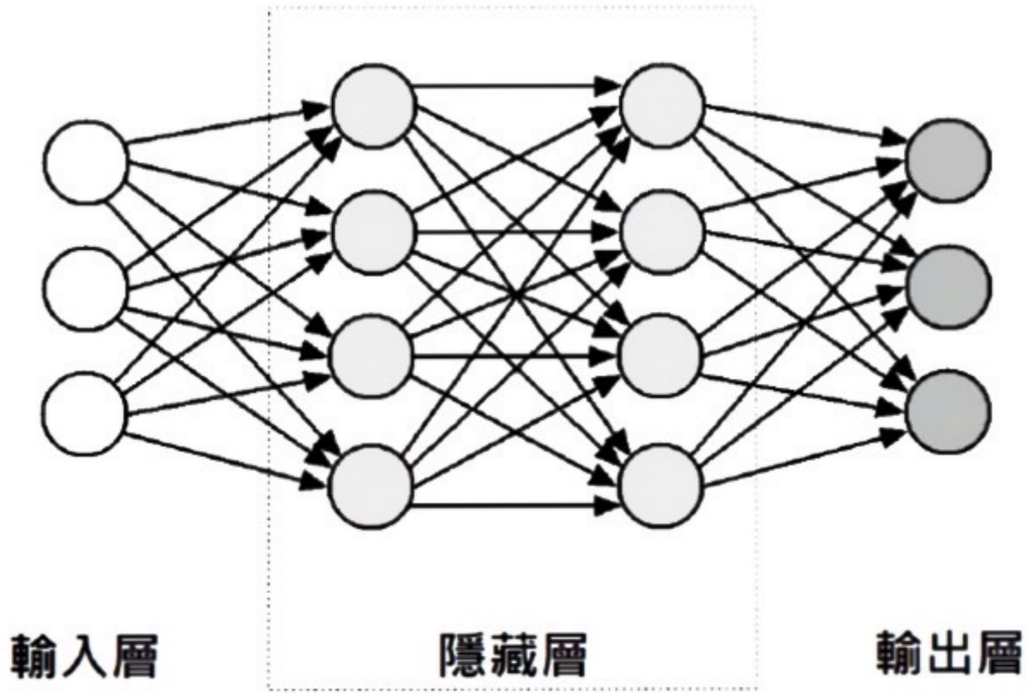
YOLO 就是一種快速且準確的物體偵測（Object Detection）演算法，也是一種深度學習演算法（Deep Learning Algorithms）。

YOLO 演算法是使用深度學習的「卷積神經網路」（Convolutional Neural Networks，CNN），如其英文名稱所述，YOLO 只需單次神經網路的前向傳播（Forward Propagation），就可以準確的偵測出多個物體。其官方網址如下所示：

```
https://pjreddie.com/darknet/yolo/
```

什麼是深度學習

深度學習就是一種機器學習，這是使用模仿人類大腦神經元（Neuron）傳輸所建立的一種神經網路架構（Neural Network Architectures），這就是深度學習演算法的核心，如下圖所示：



上述圖例是多層神經網路，每一個圓形的頂點是一個神經元，整個神經網路包含「輸入層」（Input Layer）、中間的「隱藏層」（Hidden Layers）和最後的「輸出層」（Output Layer）。

深度學習使用的神經網路稱為「深度神經網路」（Deep Neural Networks，DNNs），其中間的隱藏層有很多層，意味著整個神經網路十分的深（Deep），可能高達 150 層隱藏層。

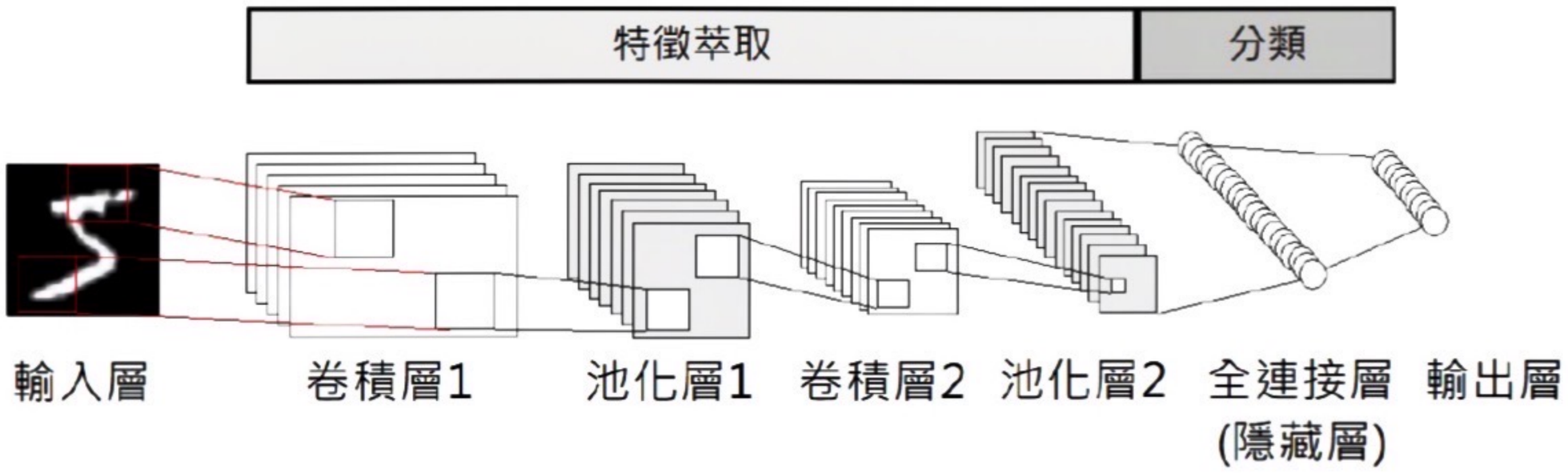


深度學習的深度神經網路是一種神經網路，早在 1950 年就已經出現，只是受限早期電腦的硬體效能和技術不純熟，傳統多層神經網路並沒有成功，為了擺脫之前失敗的經驗，所以重新包裝成一個新名稱：「深度學習」。

卷積神經網路 CNN

卷積神經網路（Convolutional Neural Network，CNN）簡稱 CNNs 或 ConvNets，其基礎是 1998 年 Yann LeCun 提出名為 LeNet-5 的卷積神經網路架構，基本上，卷積神經網路是模仿人腦視覺處理區域的神經迴路，針對影像處理的神經網路，例如：影像分類、人臉偵測和手寫辨識等。

卷積神經網路的基本結構是卷積層（Convolution Layers）和池化層（Pooling Layers），使用多種不同的神經層來依序連接成神經網路，可以執行特徵萃取和進行影像分類，如下圖所示：



YOLO 演算法就是使用卷積神經網路 CNN 的物體偵測技術，其效能超過 Fast R-CNN、Retina-Net 和 SSD（Single Shot MultiBox Detector）等其他著名的物體偵測技術。

10-1-2 Ultralytics 的 YOLO

Ultralytics 公司開發的 YOLO 模型是使用優化的 YOLO 模型結構，提供更靈活的架構來幫助開發者開發更快速、更準確且易於使用的電腦視覺解決方案，包含：物體偵測、影像分割和姿態評估等。

YOLO 模型目前仍然在持續優化和更新中，Ultralytics 公司在 2024 年 9 月 27 日釋出 YOLO11，這是繼 YOLOv8 版之後，Ultralytics 官方釋出的 YOLO 最新版本，2025 年 2 月 18 推出引入注意力機制（Attention Mechanism）的 YOLO12。

請注意！因為 Node-RED 的 yolov8 節點支援的 YOLO 物體偵測模型是 v8 版，所以本書的 YOLO 模型是使用目前最廣泛的 YOLOv8 版。在 Ultralytics 官方網站可以查詢 YOLOv8 預訓練模型，其 URL 網址如下所示：

```
https://docs.ultralytics.com/models/yolov8/
```

模型	文件名	任务	推论	验证	培训	出口
YOLOv8	yolov8n.pt yolov8s.pt yolov8m.pt yolov8l.pt yolov8x.pt	检测	✅	✅	✅	✅
YOLOv8-seg	yolov8n-seg.pt yolov8s-seg.pt yolov8m-seg.pt yolov8l-seg.pt yolov8x-seg.pt	实例分割	✅	✅	✅	✅
YOLOv8-姿势	yolov8n-pose.pt yolov8s-pose.pt yolov8m-pose.pt yolov8l-pose.pt yolov8x-pose.pt yolov8x-pose-p6.pt	姿势/关键点	✅	✅	✅	✅
YOLOv8-obb	yolov8n-obb.pt yolov8s-obb.pt yolov8m-obb.pt yolov8l-obb.pt yolov8x-obb.pt	定向检测	✅	✅	✅	✅
YOLOv8-cls	yolov8n-cls.pt yolov8s-cls.pt yolov8m-cls.pt yolov8l-cls.pt yolov8x-cls.pt	分类	✅	✅	✅	✅

上述 YOLO 預訓練模型的副檔名 .pt 是 PyTorch 格式，開發者可以依據不同應用和運算能力的裝置，專案所需執行速度和準確度來選擇使用的 YOLO 預訓練模型，其檔案格式如下所示：

```
yolo<版本><尺寸>-<任務>.pt
```

上述 yolo 後的 < 版本 > 是 YOLO 版本，YOLO12 是 12；YOLO11 是 11，YOLO8 不是 8，而是 v8，< 尺寸 > 可以是 n（Nano）、s（Small）、m（Medium）、l（Large）和 x（Extra Large）代表不同尺寸和複雜度的模型，位在「-」符號後的 < 任務 > 可以是 seg、pose、cls 和 obb，這就是【任務】欄支援的電腦視覺應用（沒有 < 任務 > 就是物體偵測和追蹤模型）。

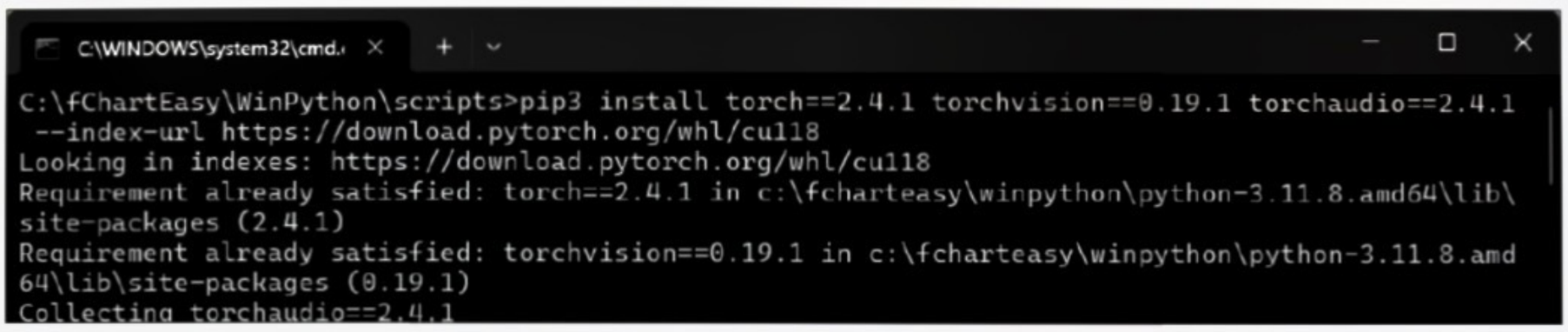
在 Python 開發環境安裝 YOLO 需要安裝 OpenCV 和 ultralytics 套件，請開啟「命令提示字元」視窗輸入下列安裝命令來進行套件安裝（fChartEasy 套件已經安裝），如右所示：

```
pip install opencv-python==4.10.0.84 Enter
pip install ultralytics==8.3.85 Enter
```

說明

YOLO 預設安裝的是 CPU 版本，GPU 版本需要支援 CUDA 的獨立顯示卡，請先更新獨顯的驅動程式後，進入 PyTorch 官網查詢安裝命令，就可以在 fChartEasy 套件執行【Python 命令提示字元 (CLI)】命令，在「命令提示字元」視窗輸入命令來安裝 GPU 版，如下所示：

```
pip install torch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1
--index-url https://download.pytorch.org/whl/cu118 Enter
```



10-2 Thonny Python IDE 的基本使用

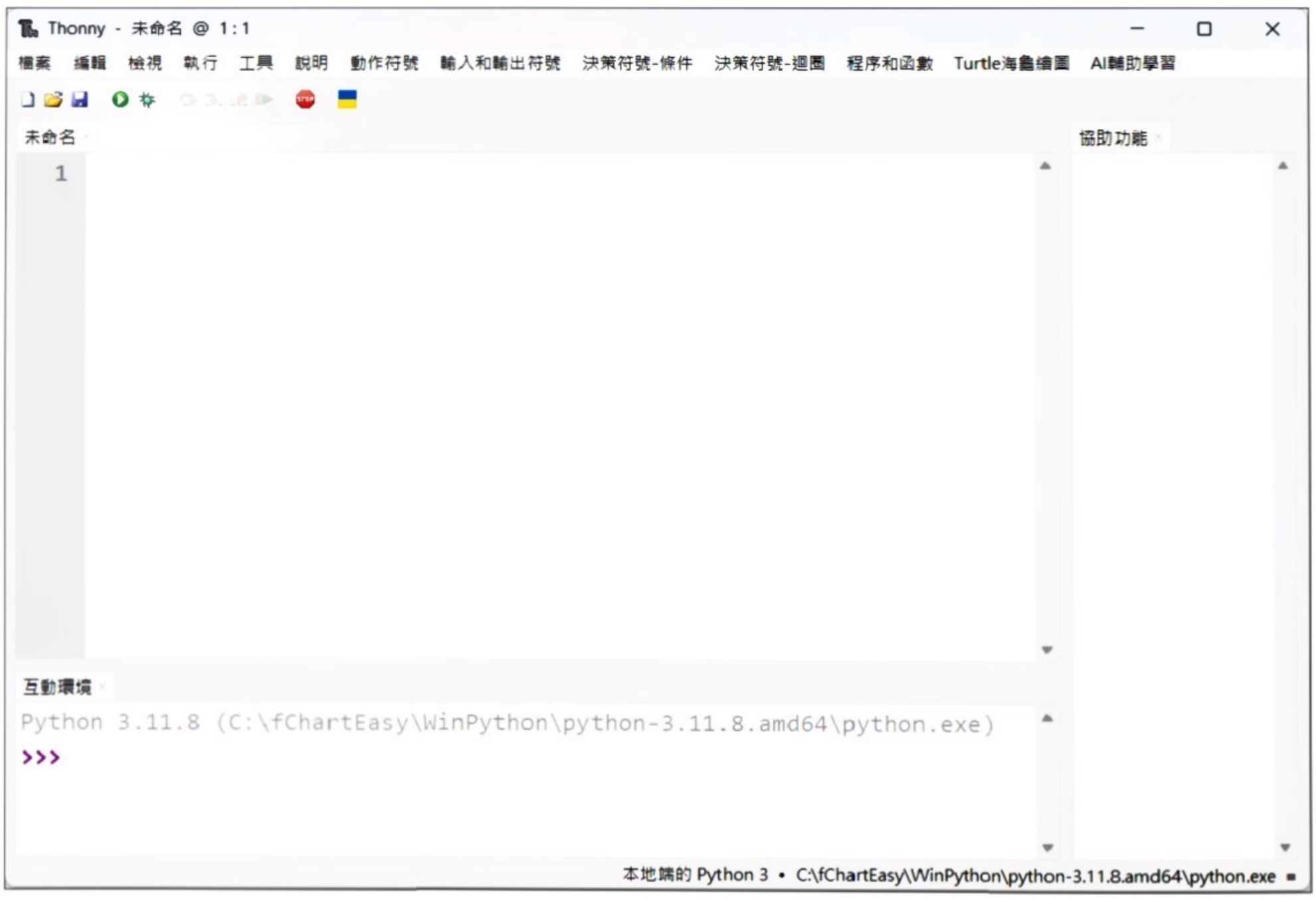
在本書使用的客製化 fChartEasy 套件已經安裝 Thonny Python IDE 和 YOLO 套件，我們可以馬上啟動 Thonny 來撰寫第 1 個 Python 程式，和在互動環境輸入和執行 Python 程式碼。

請注意！本書並不是 Python 教學書，在這一節的主要目的是讓讀者了解如何使用 Thonny 來執行 Python 程式，因為在之後的 YOLO 工具程式都是使用 Python 語言所撰寫，我們只需修改開頭的幾個變數值，就可以使用 Thonny 執行本書 YOLO 工具程式來標註和訓練你自己的 YOLO 模型。

10-2-1 建立第一個 Python 程式

現在，我們準備從啟動 Thonny 開始，一步一步建立你的第 1 個 Python 程式，其步驟如下所示：

Step 1 請在 fChart 主選單執行【Thonny Python IDE】命令啟動 Thonny 開發環境，可以看到簡潔的開發介面。



上述開發介面的上方是功能表，在功能表下方是工具列，工具列下方分成三部分，在右邊是「協助功能」視窗顯示協助說明（執行「檢視 > 協助功能」命令切換顯示），在左邊分成上 / 下兩部分，上方是程式碼編輯器的標籤頁；下方是「互動環境 (Shell)」視窗，可以看到 Python 版本 3.11.8，結束 Thonny 請執行「檔案 > 結束」命令。

Step 2 在編輯器的【未命名】標籤請輸入第一個 Python程式的程式碼，如果沒有看到此標籤，請執行「檔案 > 開新檔案」命令新增 Python 程式檔案，我們準備建立的 Python 程式只有 1 行程式碼，如下所示：

```
print("第1個Python程式")
```





請注意！如果輸入中文字串內容的 Python 程式碼，當輸入完中文字後，若無法成功輸入「"」符號，請記得從中文切換成英數模式後，即可成功輸入「"」符號。

Step 3 請執行「檔案 > 儲存檔案」命令或按工具列的【儲存檔案】鈕，然後在「另存新檔」對話方塊切換至「ch10」目錄，在下方輸入檔名【ch10-2-1】，按【存檔】鈕儲存成 ch10-2-1.py 程式。

Step 4 可以看到標籤名稱已經改成檔案名稱，然後請執行「執行 > 執行目前腳本」命令，或按工具列綠色箭頭圖示的【執行目前腳本】鈕（也可按 F5 鍵）來執行 Python 程式，就可以在下方「互動環境 (Shell)」視窗看到 Python 程式的執行結果。



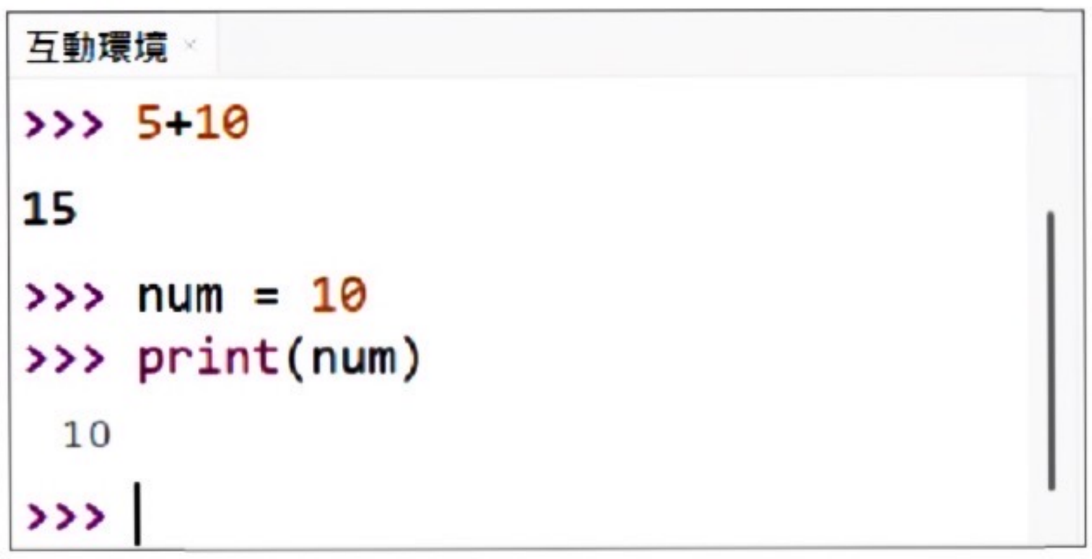
對於本書的 YOLO 相關工具程式，請執行「檔案 > 開啟舊檔」命令開啟 Python 程式檔案後，就可以馬上使用「執行 > 執行目前腳本」命令來執行 Python 程式。

10-2-2 使用 Python 互動環境

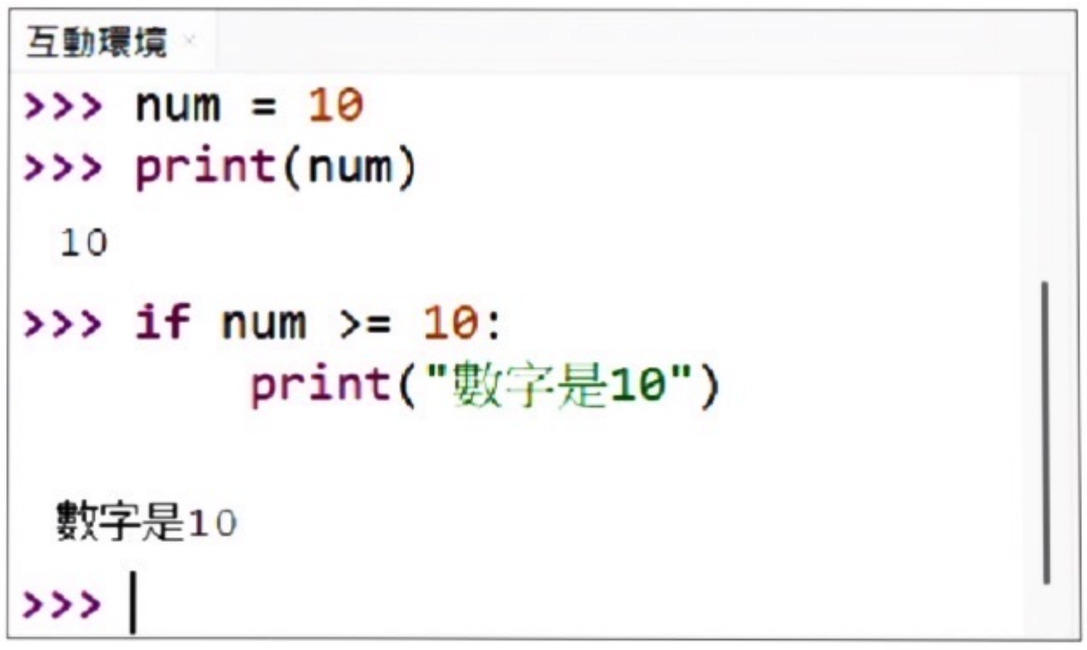
在 Thonny 開發介面下方的「互動環境 (Shell)」視窗就是 REPL 交談模式，REPL（Read-Eval-Print Loop）是循環「讀取 - 評估 - 輸出」的互動程式開發環境，可以直接在「>>>」提示文字後輸入 Python 程式碼來馬上執行程式碼。例如：輸入 5+10，按 Enter 鍵，立刻可以看到執行結果 15，如下圖所示：



同樣的，我們可以定義變數 num = 10 後，輸入 print() 函式來顯示變數 num 的值，如下圖所示：



如果輸入的是程式區塊，例如：if 條件敘述，請在輸入 if num >= 10: 後（最後輸入「:」冒號），按 Enter 鍵，就會換行且自動縮排 4 個空白字元，我們需要按二次 Enter 鍵來執行程式碼，可以看到執行結果，如下圖所示：



10-3 取得訓練 YOLO 模型的圖檔資料

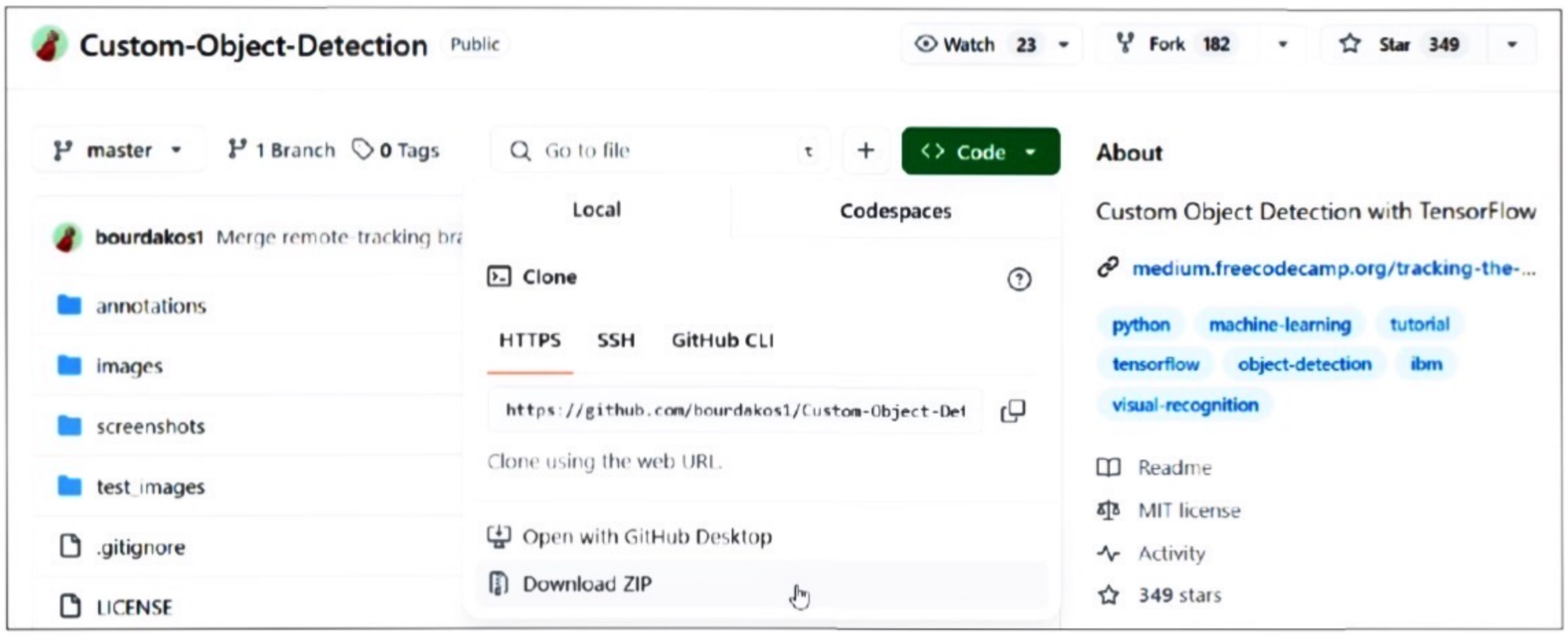
基本上，訓練 YOLO 物體偵測模型的第一步就是取得訓練模型所需的圖檔資料。一般來說，取得訓練圖檔的管道，如下所示：

- 從網路或 GitHub 自行搜尋取得訓練資料的圖檔。
- 從資料集網站下載圖檔，例如：Roboflow Universal 和 Kaggle 等。
- 使用 Python 程式從影片檔分割影格來取得訓練的圖檔資料。

10-3-1 從 GitHub 網站取得訓練資料的圖檔

在 GitHub 網站有很多訓練客製化 AI 模型的專案，我們可以直接從這些網站來取得 YOLO 訓練資料的圖檔，例如：使用星際大戰電影預告的圖檔來訓練 AI 模型的 GitHub 專案，其 URL 網址如下所示：

```
https://github.com/bourdakos1/Custom-Object-Detection/tree/master
```



當找到可用的 GitHub 專案後，請點選【Code】，執行【Download ZIP】命令，就可以將圖檔的訓練資料下載至本機電腦。在本書是解壓縮至「ch10\Custom-Object-Detection-master」目錄，其目錄結構如下圖所示：

```
Custom-Object-Detection-master
  ├─annotations
  │   └─xmls
  ├─images
  ├─screenshots
  └─test_images
```

上述「images」目錄是圖檔，「annotations」目錄是 VOC 格式的標註檔，並不是 YOLO 格式，我們可以使用程式來轉換，或自行使用第 10-4 節的 LabelImg 工具來標註圖檔。

在本章第 10-3-3 節的影片檔，就是使用此專案「images」目錄下的 JPG 圖檔轉換成的 MP4 影片檔。Python 程式：jpg_to_mp4_converter.py 只需修改第 5 行的圖檔路徑，和第 6 行的輸出影片檔名稱，就可以將指定目錄下的圖檔轉換成 MP4 影片檔，如右圖所示：

jpg_to_mp4_converter.py

```python
import cv2
import os

# 設定圖檔資料夾和輸出影片檔案名稱
image_folder = "./Custom-Object-Detection-master/images"
output_video = "starwars.mp4"
```

Python 程式的執行結果可以建立名為 starwars.mp4 的影片檔。

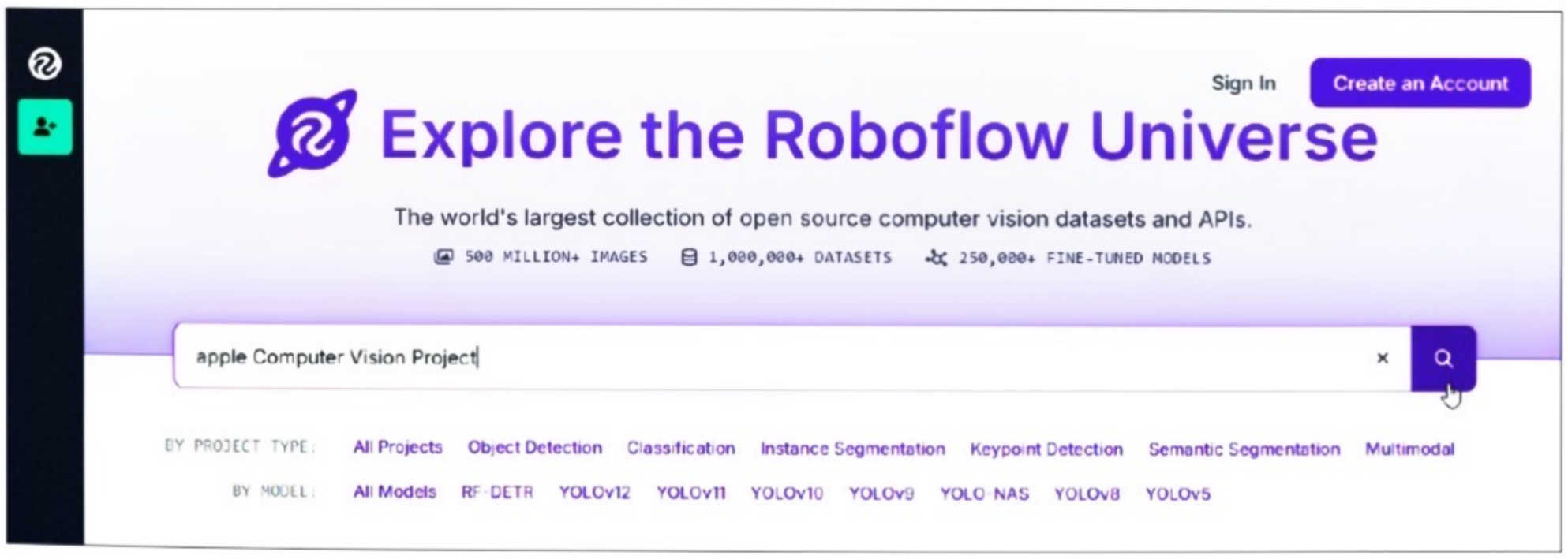
10-3-2 從 Roboflow Universal 下載訓練 YOLO 模型的資料集

Roboflow 是一個網路平台，提供完整的模型建構、訓練和部署功能，可以幫助我們管理資料集、標註影像、訓練模型和部署模型。Roboflow Universal 是 Roboflow 提供的服務，這是開放原始碼的資料集，提供超過 3.5 億的圖檔和 50 萬個資料集，其 URL 網址如下所示：

```
https://universe.roboflow.com/
```

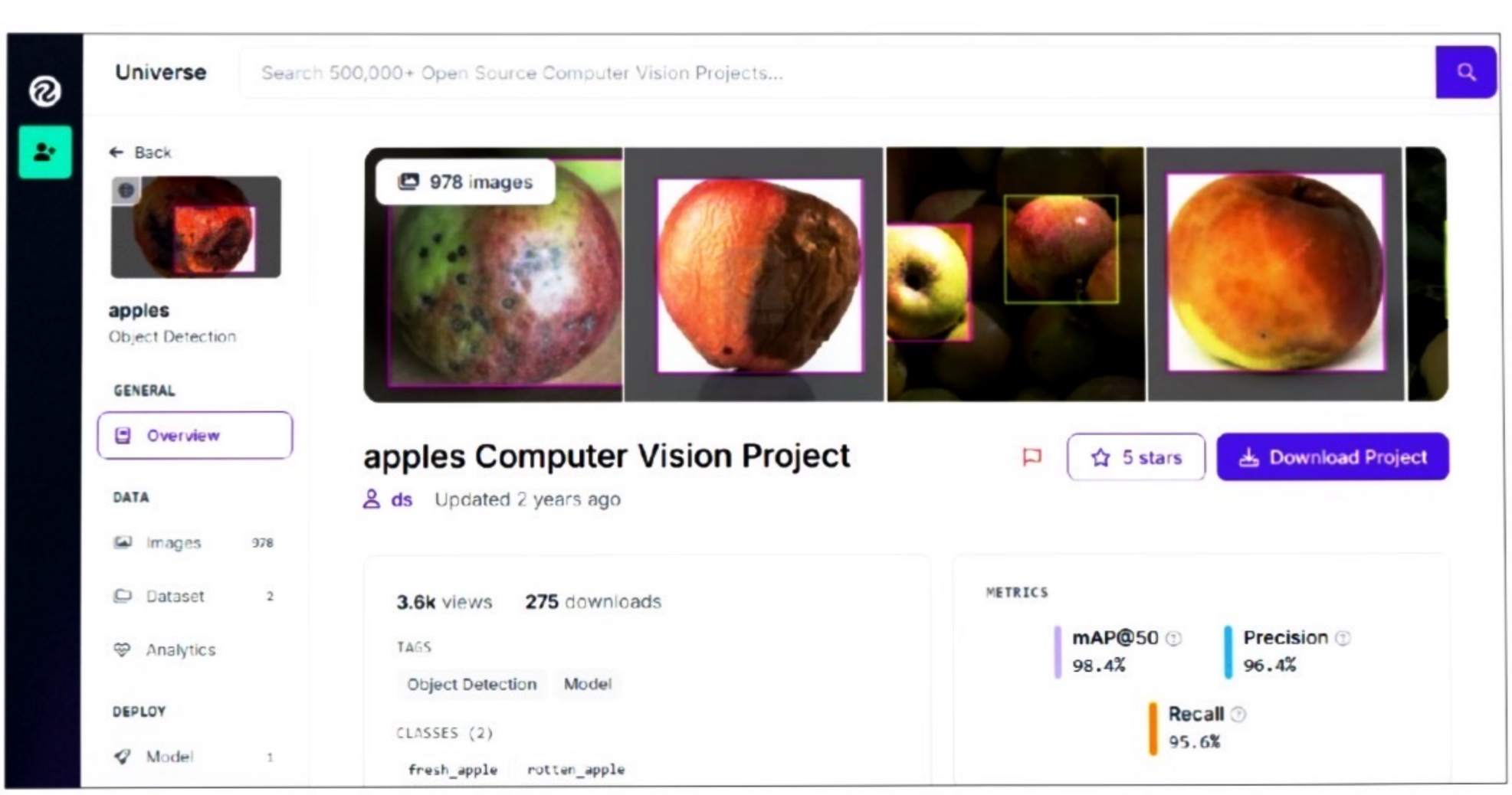
我們準備在 Roboflow Universal 搜尋和下載 apples Computer Vision Project 資料集來訓練偵測好蘋果或壞蘋果的 YOLO 物體偵測模型，其步驟如下所示：

Step 1 請啟動瀏覽器進入 Roboflow Universe 網頁後，輸入【apple Computer Vision Project】，按游標所在圖示來搜尋資料集。

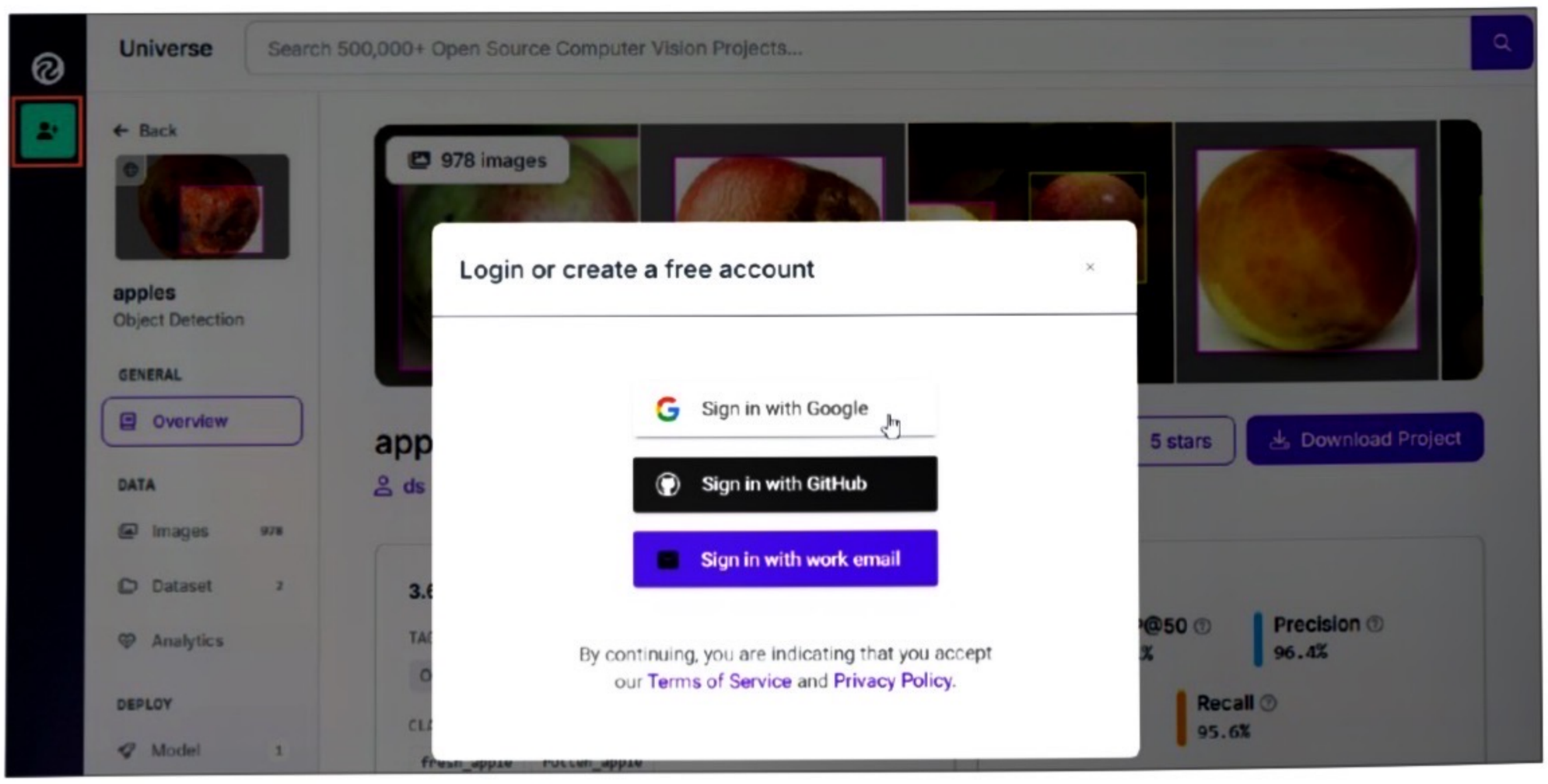


Step 2 可以看到搜尋結果找到了非常多的資料集，然後，只需點選方框就可以開啟此資料集。為了得到一致的訓練結果，本書使用的 Apple Computer Vision 資料集，其 URL 網址如下所示：

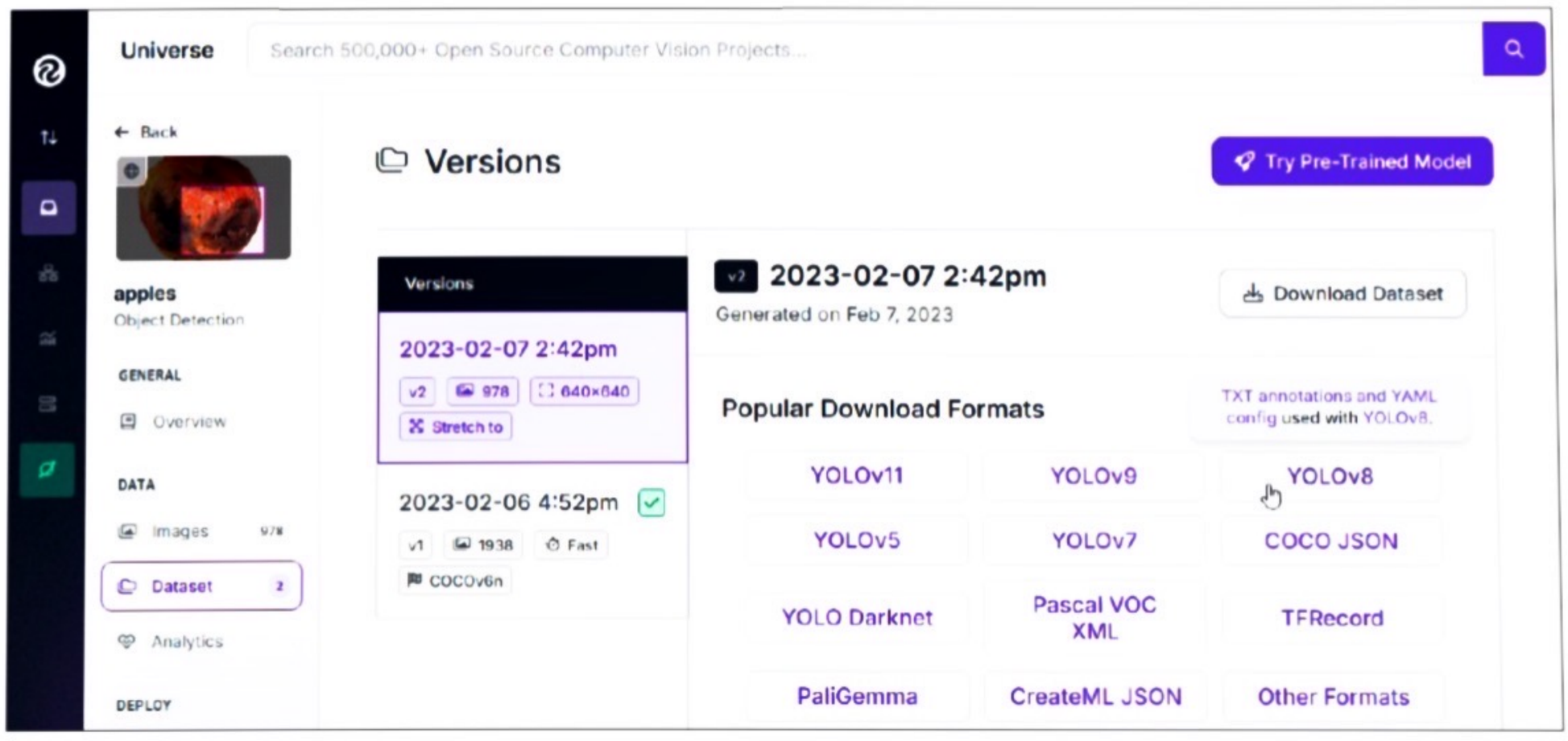
```
https://universe.roboflow.com/ds-lxa2d/apples-daz2v
```



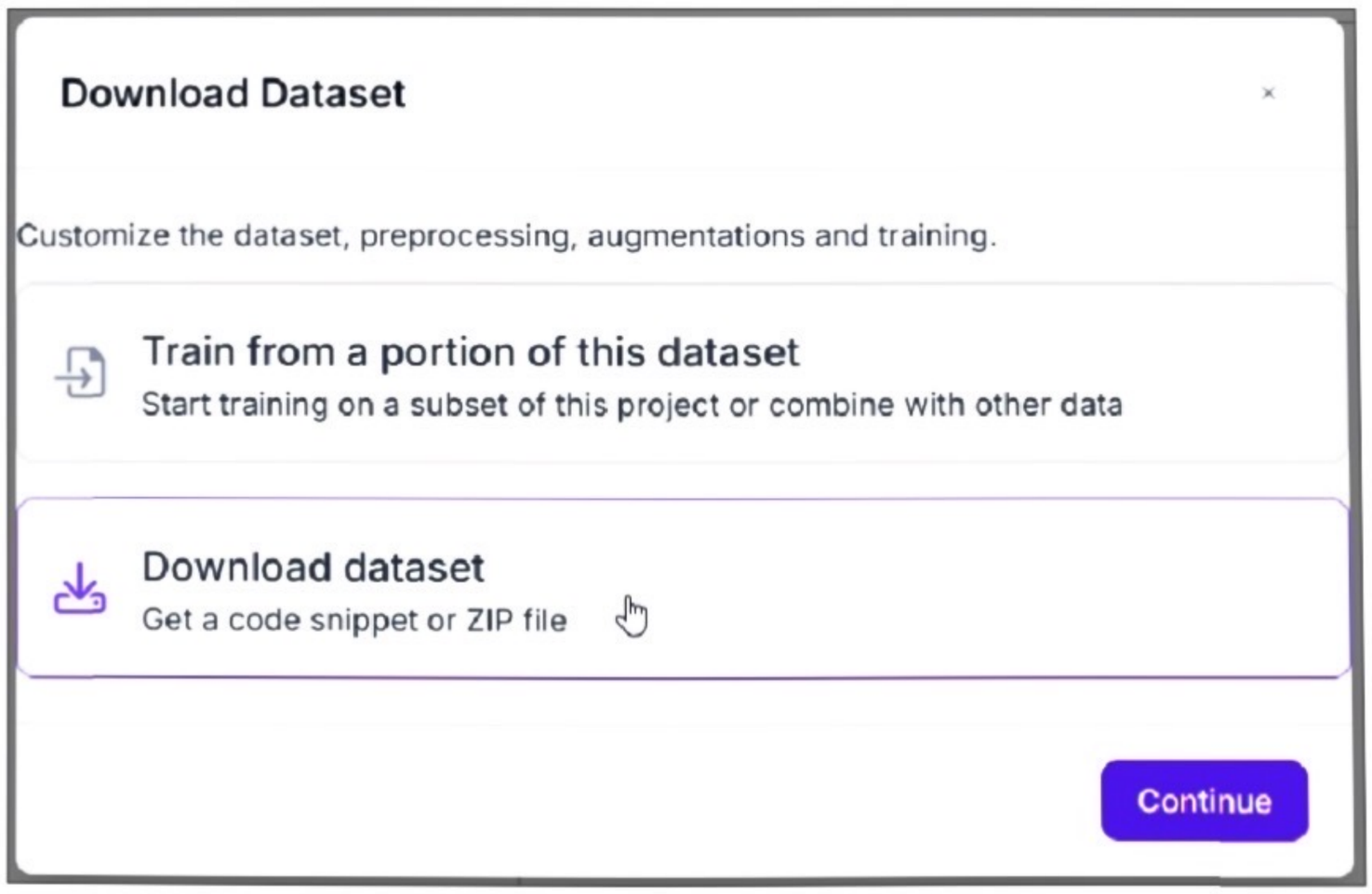
Step 3 在 Roboflow Universal 下載資料集需要登入 Roboflow，請點選左上角第 2 個圖示後，再點選【Sign in with Google】使用 Google 帳號來登入 Roboflow。



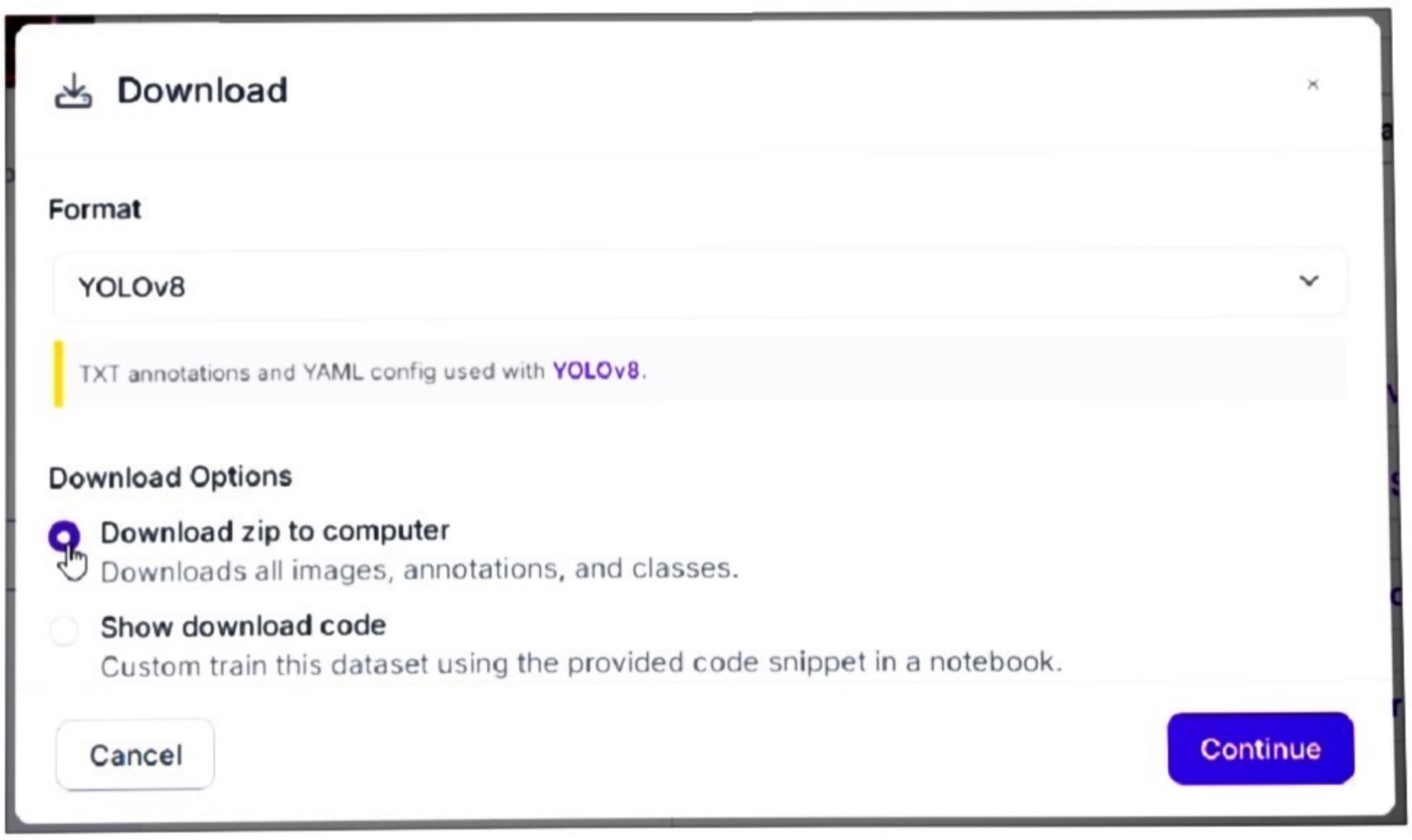
Step 4 當成功登入後，請在左邊選【Dataset】，右邊點選【YOLOv8】來下載資料集。



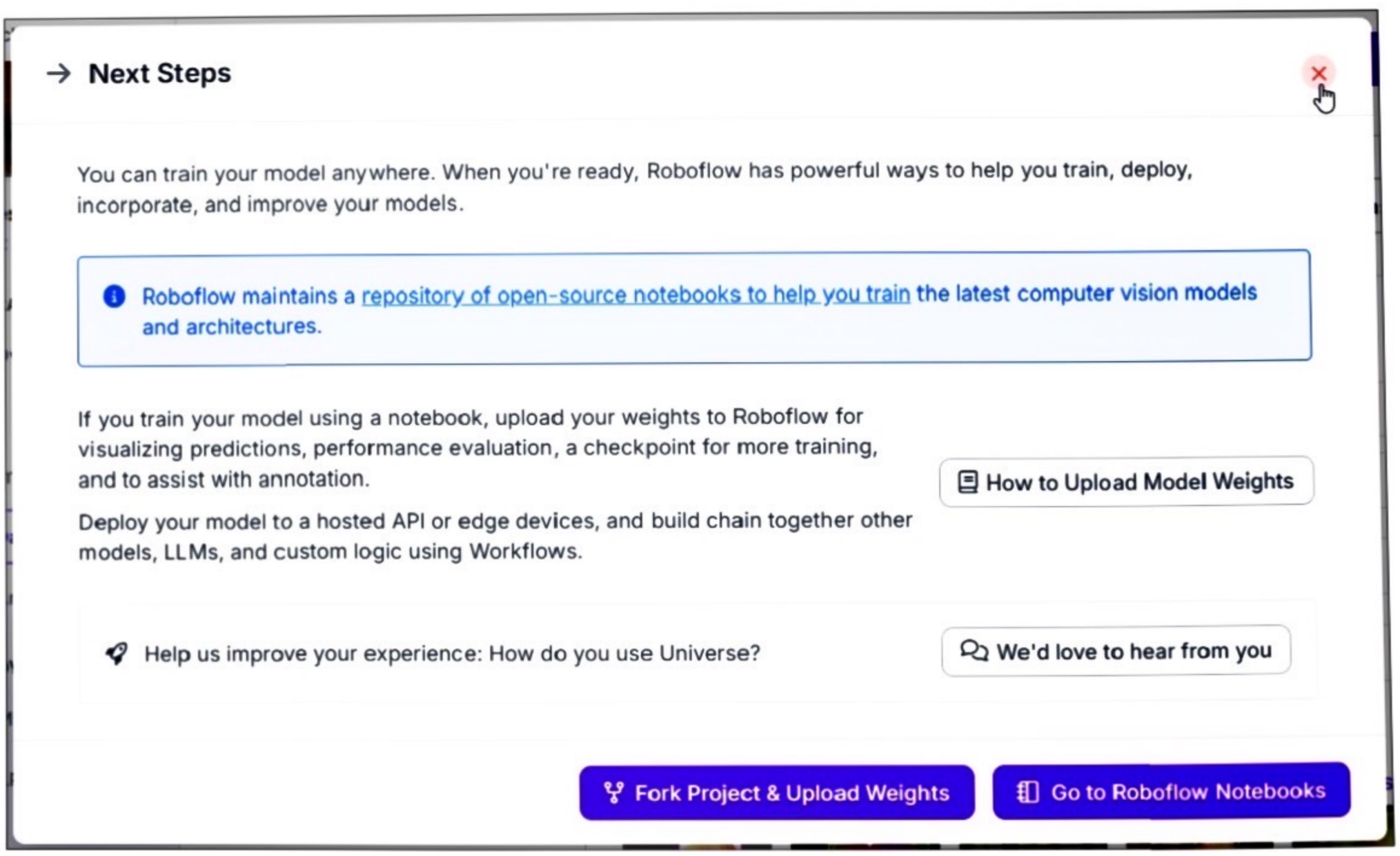
Step 5 接著選【Download dataset】後，按【Continue】鈕繼續。



Step 6 再選【Download zip to computer】，即可按【Continue】鈕來下載 ZIP 格式檔案的資料集。



Step 7 在完成下載後，請按「Next Steps」右上角的【X】圖示。



在本書下載的 YOLOv8 資料集的檔名是：apples.v2i.yolov8.zip。

10-3-3 使用視訊影片取得訓練模型的圖檔

如果擁有現成下載的 YouTube 影片檔或監控視訊的影片檔，例如：一段星際大戰電影預告的影片檔，我們就可以從視訊抽出每一張影格來另存成圖檔，然後在圖檔的影像自行標註【鈦戰機】和【千年鷹號】，就可以建立資料集，訓練偵測這 2 種戰機的 YOLO 物體偵測模型。

如果在第 10-3-1 節沒有建立 MP4 影片檔，我們也可以使用 Python 程式來下載本節所需的 MP4 影片檔。

步驟一：從網路下載 MP4 影片檔

Python 程式：ch10-3-3\step1_video_downloader.py 只需更改第 4 行影片檔的 URL 網址（url 變數）和第 6 行的輸出檔名（output_file 變數），就可以從網路下載 MP4 影片檔，如下圖所示：

step1_video_downloader.py

```
1  import requests
2
3  # 影片檔的URL網址
4  url = "https://github.com/fchart/test/raw/refs/heads/master/media/starwars.mp4"
5  # 下載的檔名
6  output_file = "starwars.mp4"
7  # ----------------------------------------
```

Python 程式的執行結果可以下載 starwars.mp4 檔案，在 Python 程式所在的「ch10-3-3」子目錄可以看到這個 MP4 檔案。

步驟二：將 MP4 影片檔的影格輸出成 JPG 圖檔

Python 程式：ch10-3-3\step2_frame_splitter.py 只需更改第 5 行的影片檔路徑（video_file 變數）和第 7 行的輸出目錄（output_dir 變數），就可以將影片檔的每一個影格抽出儲存成 JPG 圖檔，如下圖所示：

step2_frame_splitter.py

```python
import cv2
import os

# 來源影片檔的路徑
video_file = "starwars.mp4"
# 切割影格成圖檔的輸出目錄
output_dir = "frames"
# ----------------------------------------
```

Python 程式的執行結果可以在「ch10-3-3\frames」子目錄看到儲存的圖檔，如下圖所示：



說明

請注意！本書 Python 程式是使用相對路徑，在 Thonny 開啟 Python 程式檔案所在的目錄「ch10-3-3」是目前的工作目錄，output_dir 變數值 "frames" 就是在目前工作目錄「ch10-3-3」下建立「frames」子目錄，所以執行結果的圖檔是輸出至「ch10-3-3\frames」子目錄。

10-4 使用 LabelImg 標註圖檔建立訓練資料

當成功取得訓練資料的圖檔後，接著，我們就可以自行使用 LabelImg 工具來標註影像，然後執行 Python 工具程式來切割成訓練和驗證資料集，就可以建立出你自己的 YOLO 訓練資料集。

請注意！因為從 Roboflow Universal 下載的資料集已經有標註檔，請直接參閱第 11-1 節的說明，整理成第 11-2 節所需的目錄結構即可。

10-4-1 認識 LabelImg 與 YOLO 標註檔

LabelImg 是開放原始碼的影像標註工具，這是使用 Python 語言開發的工具程式，在「ch10\ch10-4-2\labelImg」目錄是中文版 LabelImg 影像標註工具，已經設定預設輸出格式是 YOLO 標註檔。在 Python 開發環境需要安裝 PyQt5 和 lxml 套件（fChartEasy 套件已經安裝），其命令如下所示：

```
pip install pyqt5==5.15.10 Enter
pip install lxml==5.3.1 Enter
```

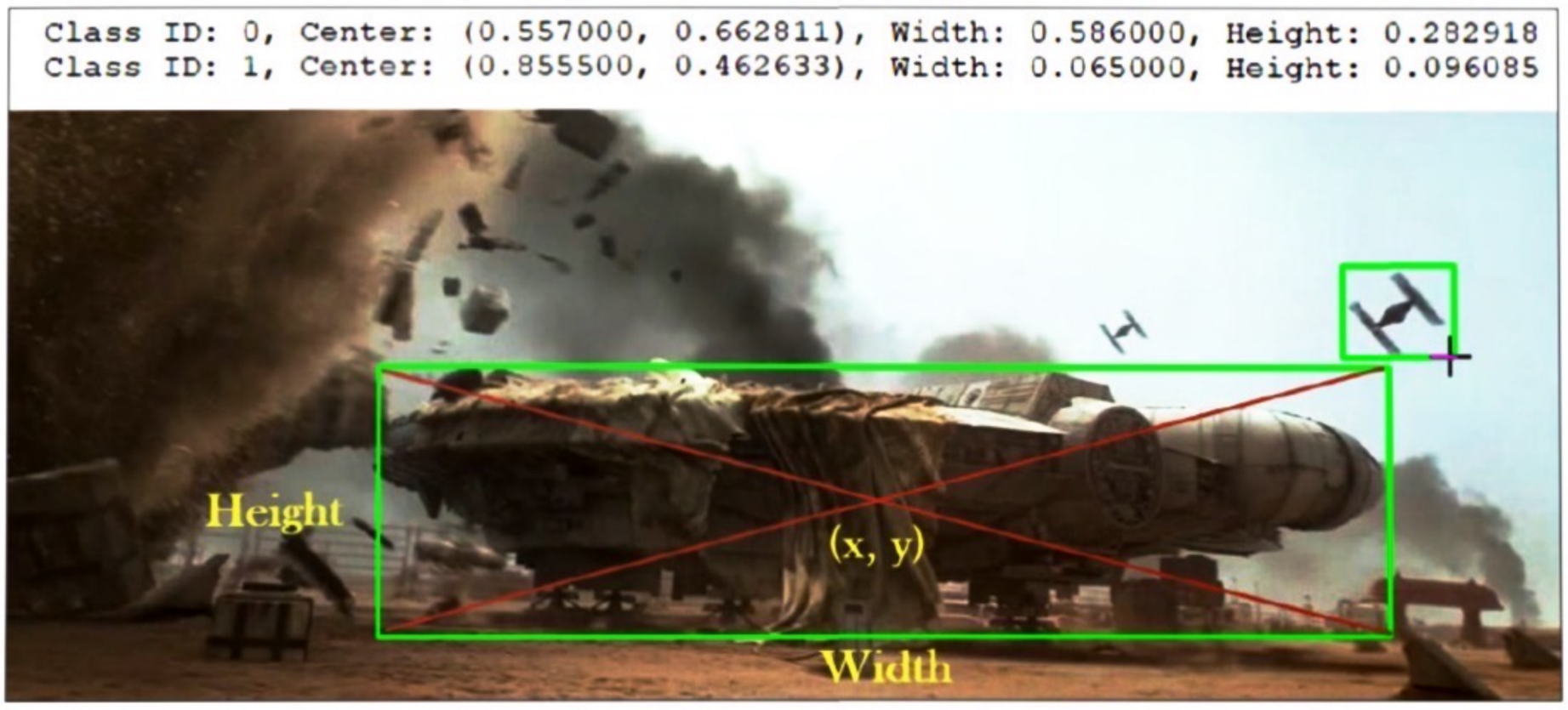
YOLO 標註檔是一個副檔名 .txt 的文字檔案，每一個圖檔對應一個同名的標註檔，例如：frame1.jpg 是對應 frame1.txt，在檔案內容的每一行標註影像中的一個物體資訊，以此例共標註 2 個物體，如下所示：

```
0 0.557000 0.662811 0.586000 0.282918
1 0.855500 0.462633 0.065000 0.096085
```

上述每一行是使用空白字元分隔成 5 項資料（數值已經分別除以影像的寬度和高度），其格式如下所示：

```
<分類索引> <中心點座標x> <中心點座標y> <邊界框的寬> <邊界框的高>
```

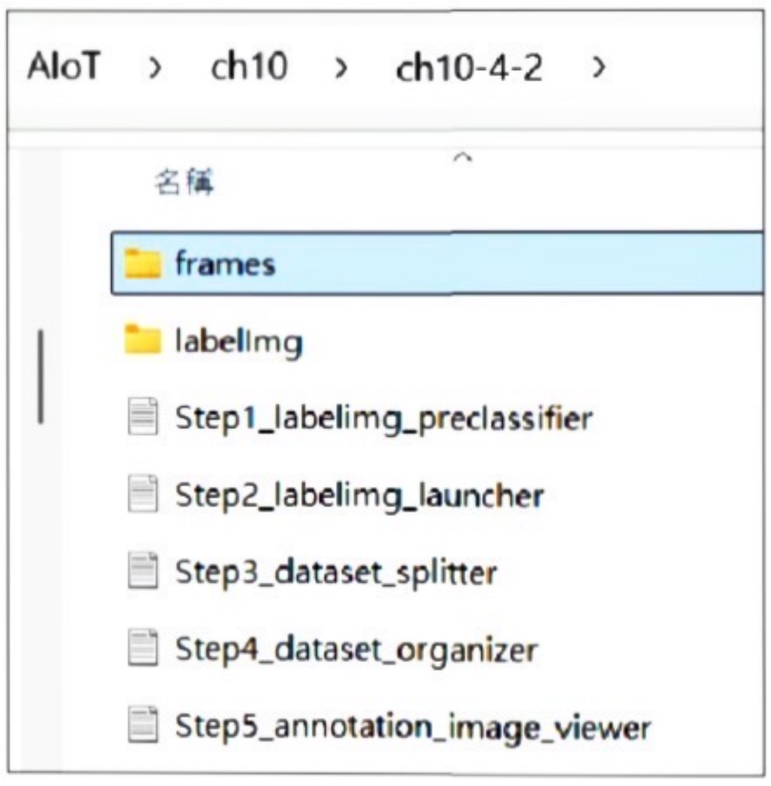
上述第 1 項資料是物體分類索引（從 0 開始），之後 2 個是中心點座標，接著是標示物體邊界框的寬度和高度，如下圖所示：



上述邊界框是用來標示出影像中欲偵測的物體的外框（在同一張影像可以標註多個物體），分類索引 0 是 "Millennium Falcon" 千年鷹號；索引 1 是 " Tie Fighter" 鈦戰機。

10-4-2 自行標註影像建立資料集

在了解 YOLO 標註檔的格式後，我們就可以啟動 LabelImg 工具來自行標註圖檔中欲偵測物體的影像，首先，請將第 10-3-3 節從影片檔抽出的「frames」圖檔目錄，複製至「ch10\ch10-4-2」目錄之下，如下圖所示：



上述「**Step1～5**」開頭的 **Python** 程式就是本節使用的工具程式，可以幫助我們建立訓練 **YOLO** 模型所需的資料集。

步驟一：設定 LabelImg 預分類資料

LabelImg 工具可以在啟動前預先設定標註影像的分類資料，這是位在「**labelImg\data**」目錄的 **predefined_classes.txt** 檔案，在此文字檔案內容的每一行就是一種分類。

Python 程式：**ch10-4-2\Step1_labelimg_preclassifier.py** 可以自動將 **Python** 串列的分類資料寫入 **predefined_classes.txt** 檔案，這是使用第 **2** 行的 **class_list** 變數定義分類串列，第 **4** 行的 **file_path** 變數指定寫入的檔案路徑，如下圖所示：

Step1_labelimg_preclassifier.py

```
1 # 定義分類串列
2 class_list = ["Millennium Falcon", "Tie Fighter"]
3 # 定義要寫入的檔案路徑
4 file_path = "labelImg/data/predefined_classes.txt"
5 # ----------------------------------------
```

Python 程式的執行結果可以建立 **predefined_classes.txt** 檔案，其內容如下圖所示：

predefined_classes.txt

```
1 Millennium Falcon
2 Tie Fighter
3
```

步驟二：啟動和使用 LabelImg 影像標註工具

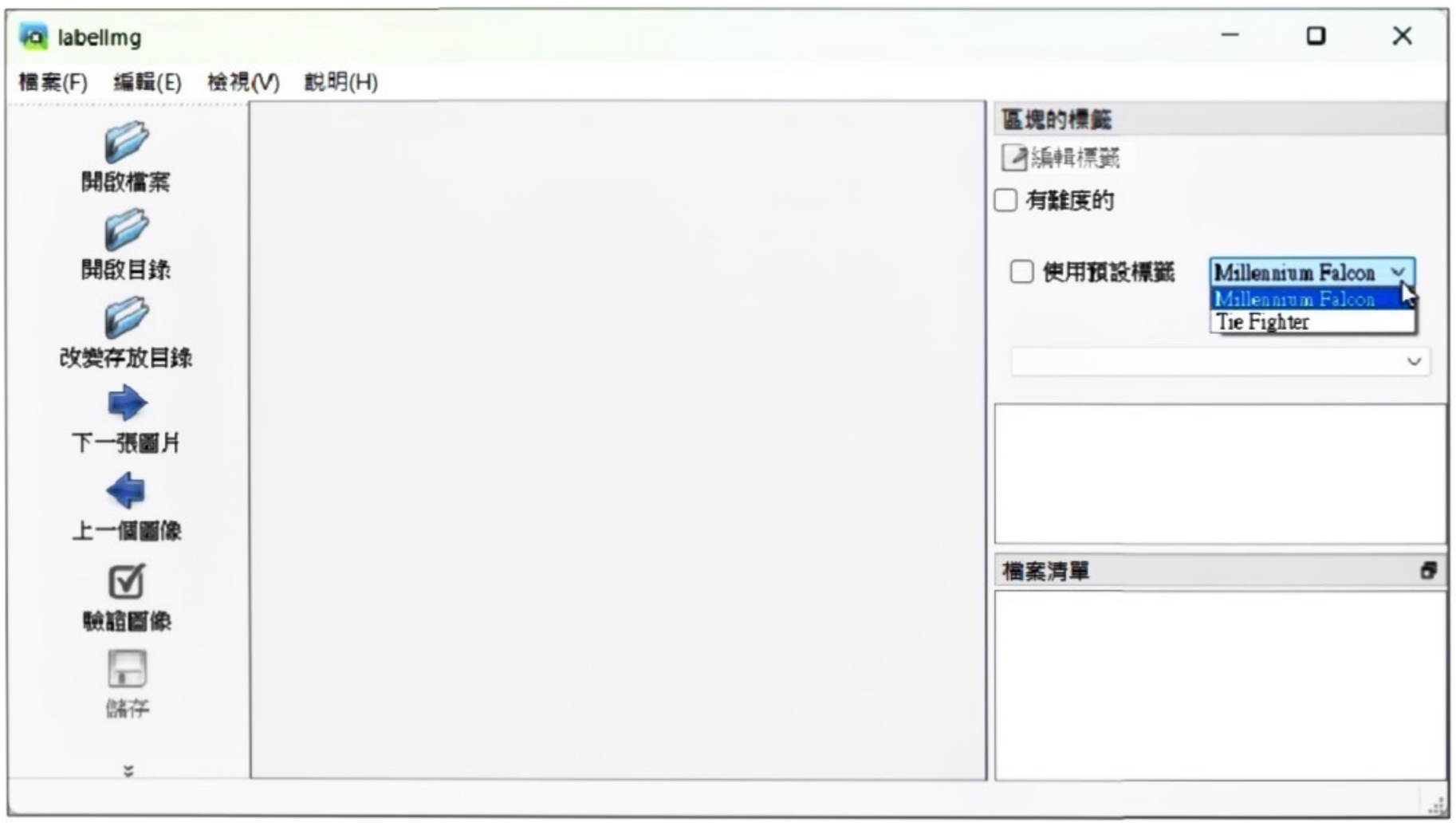
LabelImg 影像標註工具是使用 **Python** 語言所開發，我們有兩種方式來啟動此工具，如下所示：

- 在 **Thonny** 開啟「**ch10-4-2\labelImg**」目錄的 **Python** 程式 **labelImg.py**，執行此程式來啟動工具。

- 在 Thonny 執行 Python 程式：ch10-4-2\Step2_labelimg_launcher.py 來啟動 LabelImg 影像標註工具。

當成功啟動 LabelImg 後，就可以使用 LabelImg 影像標註工具來標註「frames」目錄下的圖檔，其步驟如下所示：

Step 1 請使用前述任一種方法來啟動 LabelImg 影像標註工具後，在右邊「區塊的標籤」視窗可以看到【使用預設標籤】欄位，在之後的下拉式選單，就是預先載入的 2 種分類，如下圖所示：



Step 2 執行「檔案 > 開啟目錄」命令（或點選【開啟目錄】），選擇「ch10\ch10-4-2\frames」目錄，即可顯示此目錄的第 1 張圖檔，按 Ctrl 和 + 鍵是放大影像；Ctrl 和 - 鍵是縮小影像，在右下方視窗可以看到此圖檔目錄下的檔案清單，如右圖所示：

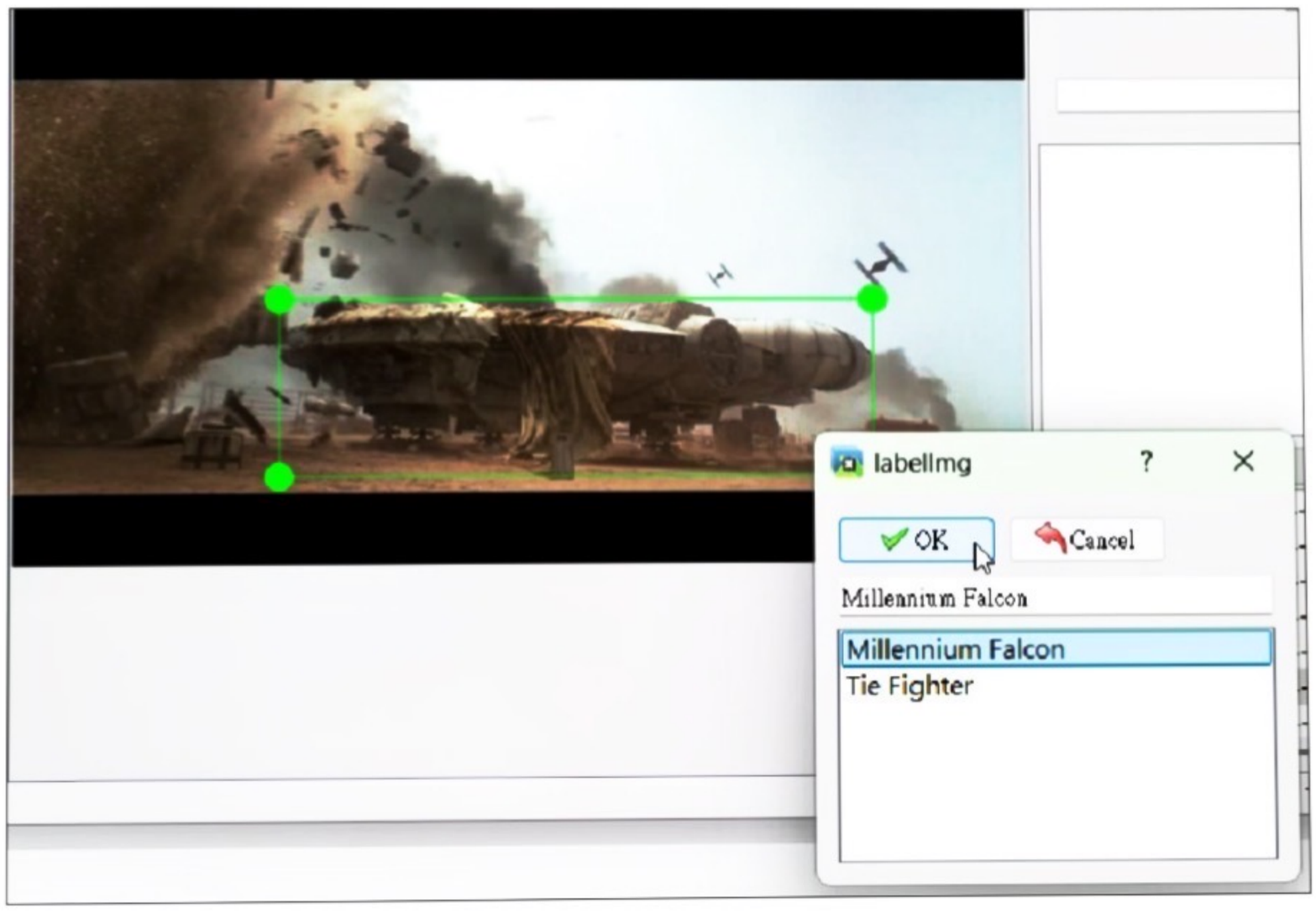


Step 3 在側邊欄點選【下一張圖片】是切換至下一個圖檔；【上一張圖像】是上一個圖檔，因為並不是每一個影格都有欲標註的物體，請切換至 fram65.jpg，或請直接雙擊右下角「檔案清單」視窗的檔名來切換至此圖檔。

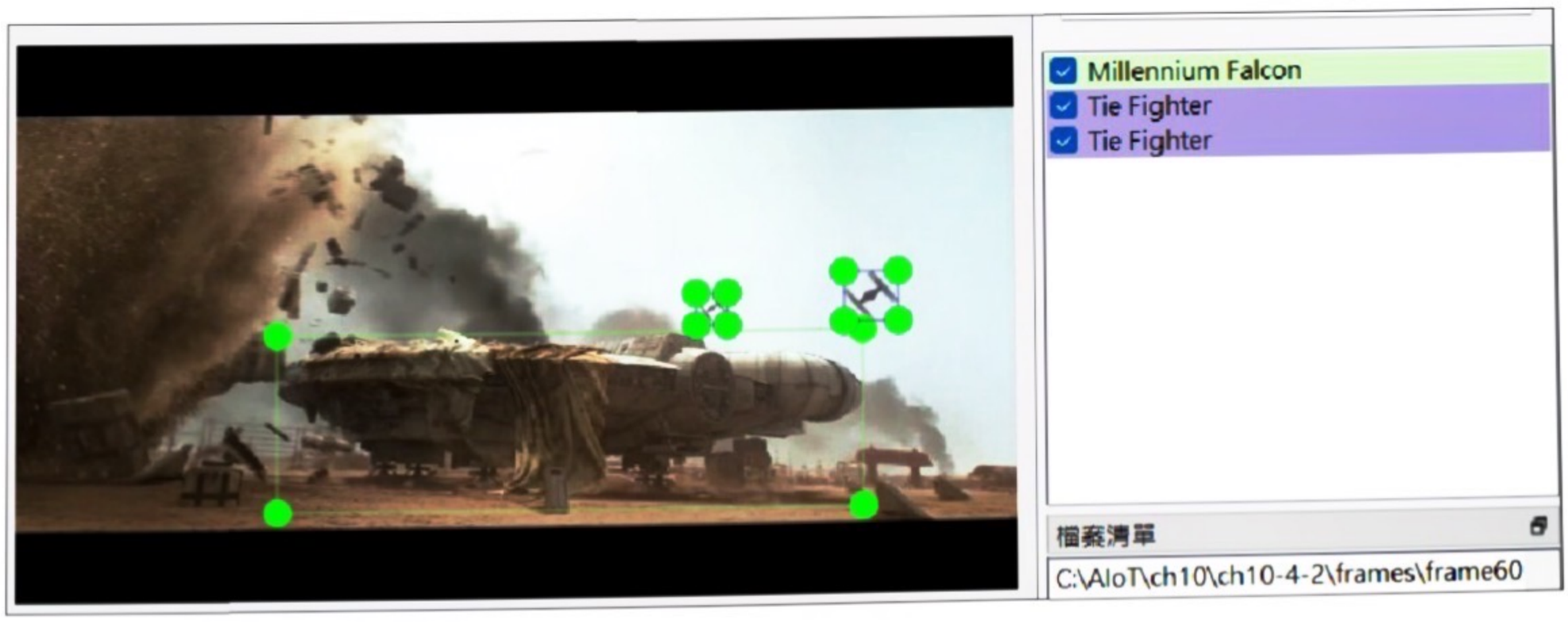
Step 4 在開啟欲標註的圖檔後，請在側邊欄選【創建區塊】，就可以在中間編輯區域從左上角至右下角拖拉出包圍物體的邊界框。



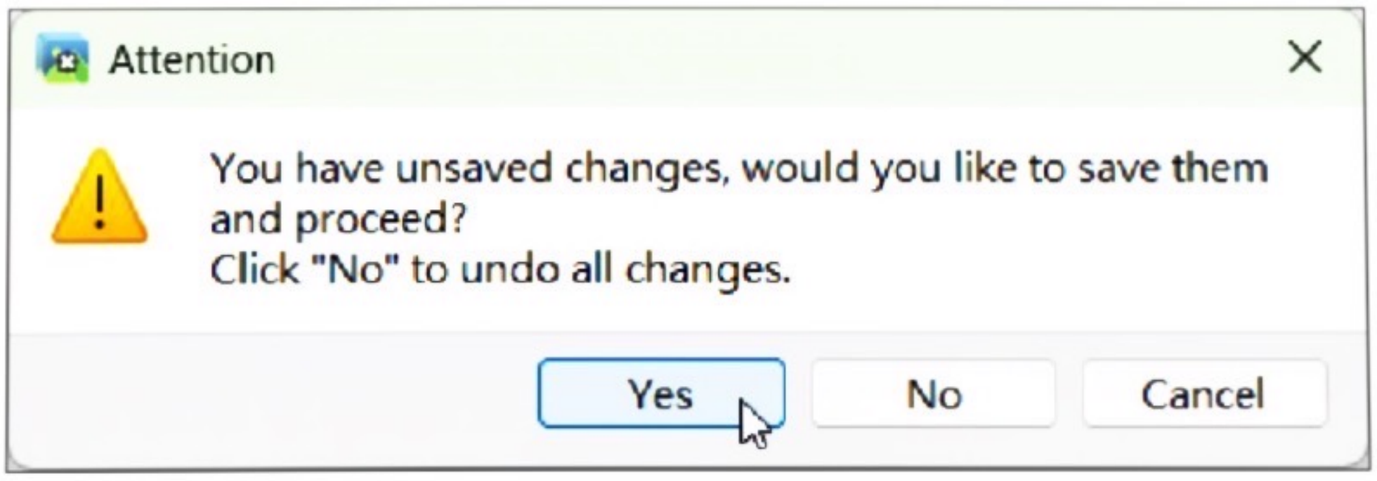
Step 5 當放開滑鼠左鍵，就可以看到分類視窗，請選【Millennium Falcon】，按【OK】鈕。



Step 6 請重複操作 2 次，先選【創建區塊】，再一一拖拉出上方 2 台【Tie Fighter】和選擇分類後，就可以看到我們已經在影像上共標註了 3 個物體，如下圖所示：



Step 7 在完成圖檔標註後，請執行「檔案 > 儲存」命令儲存標註檔，如果是選【下一張圖片】，因為尚未儲存標註檔，就會顯示尚未儲存的訊息視窗，請按【Yes】鈕儲存標註檔。



Step 8 當切換至下一個圖檔後，就可以重複上述步驟來標註物體，直到所有影像標註完成。

LabelImg 影像標註工具預設就會自動在「frames」目錄下建立名為 classes.txt 的分類檔案。

步驟三：分割成訓練和驗證資料集

在完成整個「frames」目錄的圖檔標註後，我們需要將資料集分割成訓練和驗證資料集。Python 程式：ch10-4-2\Step3_dataset_splitter.py 是一個檔案處理程式，我們只需修改第 5～10 行的變數，就可以將指定目錄下的檔案依比例分割成訓練資料集和驗證資料集，如下圖所示：

Step3_dataset_splitter.py

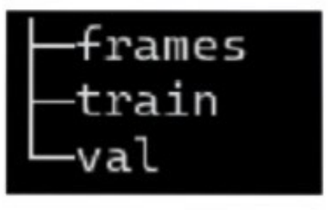
```
1  import os
2  import shutil
3  import random
4
5  input_dir = "frames"  # 請將此處替換為您的輸入目錄
6  train_dir = "train"   # 訓練資料集目錄
7  val_dir = "val"       # 驗證資料集目錄
8  split_ratio = 0.8     # 訓練與驗證的分割比例
9  img_type = ".jpg"     # 圖檔類型
10 operation = "copy"    # 選擇 "move" 搬移檔案 或 "copy" 複製檔案
11 # ----------------------------------------
```

上述 input_dir 變數是 LabelImg 標註圖檔的目錄，train_dir 和 val_dir 變數分別是訓練資料集和驗證資料集的輸出目錄，split_ratio 變數是分割比例，0.8 就是 80% 是訓練資料集；20% 是驗證資料集。

Python 程式的執行結果，可以看到已經使用分割比例 0.8，和 copy 複製操作分割成「train」和「val」目錄的訓練資料集和驗證資料集，如下所示：

```
>>> %Run Step3_dataset_splitter.py
使用 copy 操作處理 frames 目錄的資料集
資料集分割成 train 和 val 目錄的資料集
使用的分割比例: 0.8
```

在「ch10\ch10-4-2」目錄可以看到新增的 2 個目錄，如下圖所示：



步驟四：將資料集再分割成圖檔和標註檔資料夾

YOLO 資料集的目錄結構還需要將圖檔和標註檔分別置於之下的「images」和「labels」子目錄，如下圖所示：



現在，我們需要進一步將資料集再分割成圖檔和標註檔的 2 個子目錄。Python 程式：ch10-4-2\Step4_dataset_organizer.py 是一個檔案處理程式，只需修改第 4～5 行的變數，就可以重組目錄結構成為 YOLO 資料集的目錄結構，如右圖所示：

Step4_dataset_organizer.py

```
1  import os
2  import shutil
3
4  train_dir = "train"    # 訓練資料集目錄
5  val_dir = "val"        # 驗證資料集目錄
6  # ----------------------------------------
```

上述 **train_dir** 和 **val_dir** 變數分別是訓練資料集和驗證資料集的目錄，Python 程式的執行結果，可以看到目錄下的檔案已經分別分配至「**images**」和「**labels**」子目錄，如下所示：

```
>>> %Run Step4_dataset_organizer.py
  目錄的檔案已經分配至 'train\images' 和 'train\labels' 目錄
  目錄的檔案已經分配至 'val\images' 和 'val\labels' 目錄
```

在「**ch10\ch10-4-2**」目錄可以看到目前的目錄結構，如下圖所示：

```
├─frames
├─train
│  ├─images
│  └─labels
└─val
   ├─images
   └─labels
```

上述「**train**」和「**val**」目錄需要複製至第 **11-2** 節的「**data**」目錄下，而第 **11-2** 節的「**data**」目錄就是訓練 YOLO 模型的資料集根目錄。

步驟五：瀏覽 LabelImg 標註的圖檔

當成功建立 YOLO 資料集後，Python 程式：**ch10-4-2\ Step5_annotation_image_viewer.py** 可以瀏覽我們標註的 YOLO 資料集，我們只需修改第 **4～6** 行的變數值，就可以使用 OpenCV 開啟圖檔來顯示標註的邊界框和分類，如下圖所示：

Step5_annotation_image_viewer.py

```python
import cv2
import os

image_folder = "train/images"     # 圖檔路徑
label_folder = "train/labels"     # 標註檔路徑
class_names = ["Millennium Falcon", "Tie Fighter"]
# ----------------------------------------
```

上述 image_folder 變數是圖檔路徑；label_folder 是標註檔路徑，以此例是訓練資料集，class_names 變數是分類名稱串列。

Python 程式的執行結果，可以顯示圖檔影像和標示出物體邊界框與上方的分類名稱，如下圖所示：

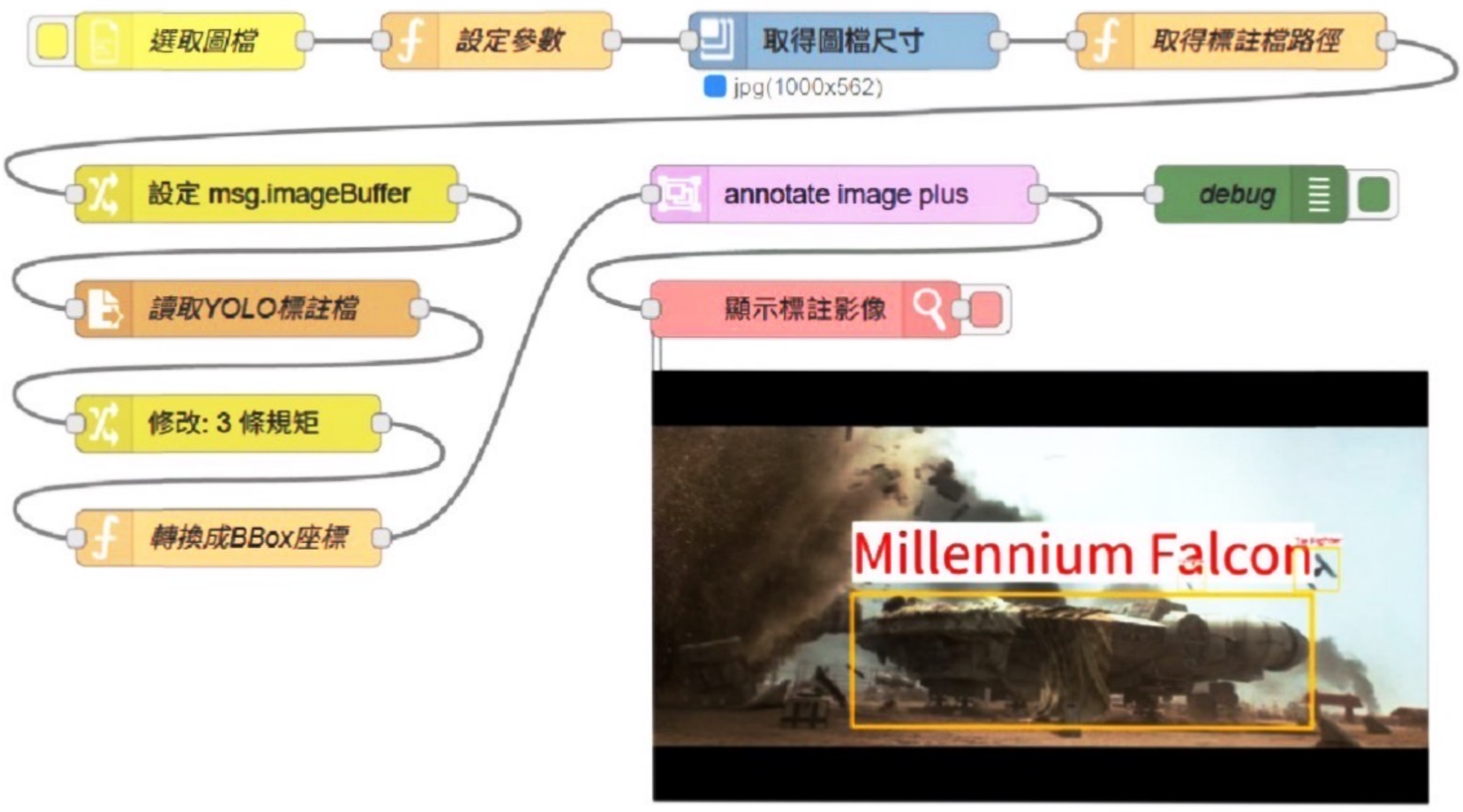


請按任意鍵，就可以顯示下一張圖檔和標註資料，直到沒有圖檔為止，請按 Esc 鍵離開。

10-5 整合應用：在 Node-RED 顯示標註圖檔

如同上一節步驟五的 Python 程式，我們一樣可以建立 Node-RED 流程來顯示標註的圖檔。在 Node-RED 需要在【節點管理】安裝 node-red-contrib-image-info 節點來取得圖檔尺寸，以便計算出真正的座標。

Node-RED 流程：ch10-5.json 是在【設定參數】的 function 節點指定分類和圖檔 images 的上一層路徑，就可以點選 file inject 節點開啟圖檔來顯示標註內容，其執行結果如下圖所示：



Node-RED 流程的節點說明，如下所示：

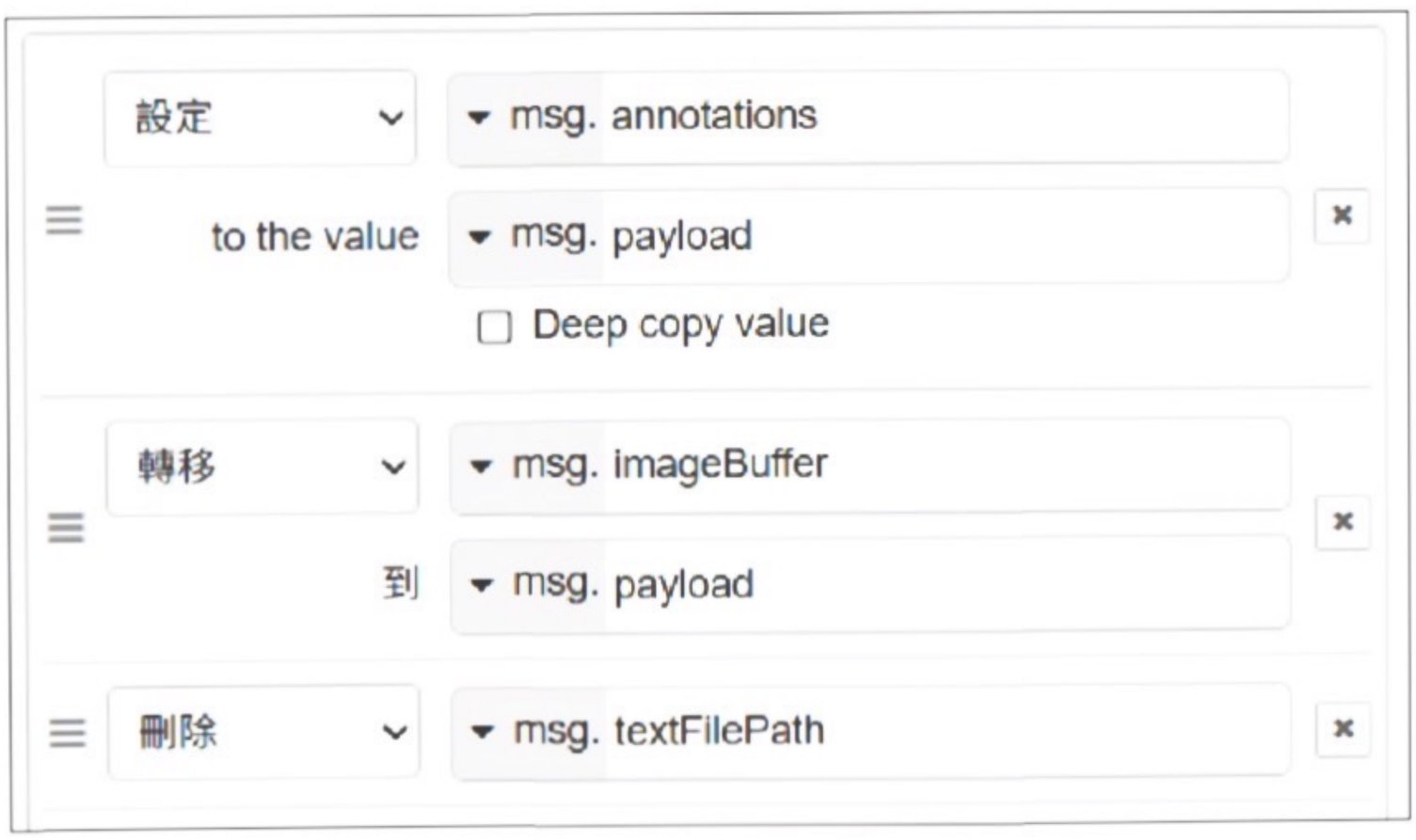
- file inject 節點：名為【選取圖檔】的節點是用來選取圖檔。
- function 節點：名為【設定參數】的節點是用來指定分類陣列和圖檔路徑，圖檔路徑是在「images」目錄的上一層，在最後需要加上「/」(請注意！因為權限問題，在瀏覽器選取的圖檔並無法取得完整的檔案路徑，只能取得圖檔名稱，所

以我們需要指定圖檔所在的絕對路徑，以便自行建立選取圖檔的路徑），如下圖所示：

```
// 分類名稱陣列
msg.classNames = ["Millennium Falcon", "Tie Fighter"];
// 圖檔資料集的絕對路徑, 最後需加上"/", 如用"\", 請用"\\"
msg.filePath = "C:/AIoT/ch10/ch10-4-2/train/";

return msg;
```

- image-info 節點：名為【取得圖檔尺寸】的節點可以取得選取影像的尺寸，即 msg.width 的寬和 msg.height 的高。
- function 節點：名為【取得標註檔路徑】的節點是使用 JavaScript 程式碼依據選取圖檔的檔名和 msg.filePath 來建立對應 YOLO 標註檔的完整路徑，即 msg.textFilePath。
- change 節點：使用 msg.imageBuffer 保留 msg.payload 的影像資料。
- read file 節點：名為【讀取 YOLO 標註檔】的節點可以讀取 msg.textFilePath 路徑的標註檔，輸出是一個 Buffer 物件。
- change 節點：三條規則分別指定 msg.annotations 的值是 msg.payload，就可以使用 msg.imageBuffer 回存 msg.payload 影像資料，最後刪除 msg.textFilePath（已經用不到），如下圖所示：



- **function 節點**：名為【轉換成 BBox 座標】的節點是參考上一節步驟五的 Python 程式，只是改用 JavaScript 程式碼依據讀取的標註檔來轉換成【annotate image plus】節點所需 msg.annotations 屬性的註記資料，這是一個 JSON 物件。
- **annotate image plus 節點**：使用 YOLO 註記檔轉換成的 BBox 座標來標註圖檔的影像。
- **viewer 節點**：名為【顯示標註影像】的節點可以顯示標註結果的影像，【Image】欄是影像資料來源，【Width】欄可以指定影像寬度，高度會自動依比例來調整，如下圖所示：



學習評量

1. 請簡單說明什麼是深度學習和卷積神經網路 CNN ？什麼是 Ultralytics 公司開發的 YOLO ？
2. 請問 Thonny Python IDE 是什麼？什麼是 Roboflow、Roboflow Universal 和 LabelImg 工具？
3. 請自行搜尋 Roboflow Universal 找到一個資料集，然後下載 YOLOv8 版的資料集。
4. 請從 https://www.pexels.com/zh-tw/ 網站搜尋影片檔後，參閱第 10-3-3 節的步驟來下載和分割成模型訓練所需的圖檔。
5. 請繼續學習評量 4.，使用第 10-4-2 節的步驟自行使用 LabelImg 工具來標註圖檔，並且建立訓練 YOLO 模型所需的資料集。